



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Practica 1:

Solucion a una ecuacion cuadratica de 2do y 3er orden.

Integrantes:

Espinoza Matamoros Percival Ulises - 320025561

Flores Colin Victor Jaziel - 320266083

Garcia Cortes Adolfo de Jesus - XXXXXX

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - XXXXXX

Microcomputadoras

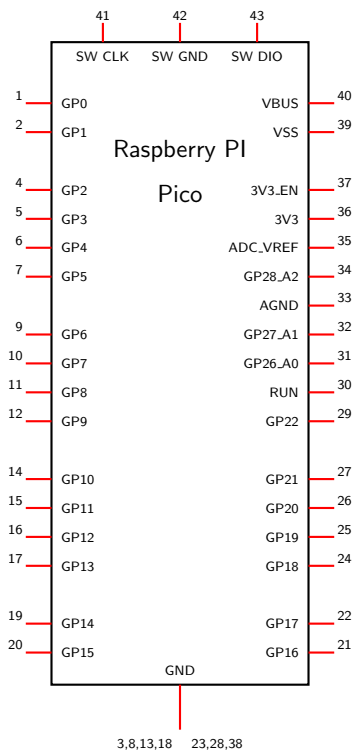
Grupo: 03 - Semestre: 2026-2

Calf: _____



Mediante la creacion de codigo en MicroPython, dar solución a una ecuación cuadrática de segundo y tercer orden utilizando la Raspberry Pi Pico, donde los coeficientes de la ecuación se ingresarán a través de la terminal, y el resultado se mostrará en la misma terminal.

Para la implementacion de este proyecto se utiliza la Raspberry Pi Pico, cuyo diagrama de pines se muestra a continuacion:



Para resolver una ecuacion cuadratica de segundo orden, tenemos la siguiente formula general:

Donde a , b y c son los coeficientes de la ecuación, y x es la variable que queremos encontrar.



Para resolver esta ecuacion, utilizamos la formula cuadratica:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (2)$$

Esta ecuacion es vista en la funcion `resolver_cuadratica`, donde se reciben los coeficientes a , b y c como argumentos, y se devuelve una tupla con las dos soluciones de la ecuacion.

Listing 1: Función para resolver ecuaciones cuadráticas

```
1 def resolver_cuadratica(coef_a, coef_b, coef_c, mostrar_pasos=False):
2     if coef_a == 0:
3         return None, "Error: No es una ecuacion de segundo grado (a=0)."
```

4

```
5     discriminante = coef_b ** 2 - 4 * coef_a * coef_c
6     raiz_discriminante = cmath.sqrt(discriminante)
7
8     raiz_1 = (-coef_b + raiz_discriminante) / (2 * coef_a)
9     raiz_2 = (-coef_b - raiz_discriminante) / (2 * coef_a)
10
11     if mostrar_pasos:
12         mostrar_proceso_cuadratica(coef_a, coef_b, coef_c, discriminante,
13                                     raiz_discriminante, raiz_1, raiz_2)
14
15     return (raiz_1, raiz_2), None
```

Es que la funcion recibe los coeficientes de la ecuacion cuadratica, y primero verifica si a es igual a cero, lo cual indicaria que no es una ecuacion de segundo grado. Despues de esto se calcula el discriminante $b^2 - 4ac$, y su raiz cuadrada utilizando la libreria `cmath` para manejar numeros complejos, la razon de utilizar esa libreria en la pico es por que el discriminante puede ser negativo, lo que resultaria en soluciones complejas, y la pico no tiene una funcion de raiz cuadrada que maneje numeros complejos, por lo que se utiliza `cmath.sqrt` para asegurar que se puedan calcular las raices incluso cuando el discriminante es negativo.

Por ultimo se calcula las dos soluciones utilizando la formula cuadratica, tanto la raiz positiva como la negativa, este resultado se devolvera como una tupla, no obstante, tambien tenemos el caso de si el usuario quiere ver los pasos detallados y no simplemente la solucion, para eso, si el usuario quiere ver los pasos, entonces se entra en la funcion `mostrar_proceso_cuadratica`, la cual recibe los argumentos de todos los coeficientes y las raices resultantes.

La funcion `mostrar_proceso_cuadratica` se encarga de imprimir en la terminal el proceso de como se llego a la solucion, donde el bloque de codigo es el siguiente:



Listing 2: Función para mostrar el proceso de resolución de la ecuación cuadrática

```
1 def mostrar_proceso_cuadratica(coef_a, coef_b, coef_c, discriminante, raiz_discriminante
2     , raiz_1, raiz_2):
3     ecuacion = construir_ecuacion_cuadratica(coef_a, coef_b, coef_c)
4     print()
5     imprimir_separador()
6     print(" PROCESO DE CALCULO")
7     imprimir_separador()
8     print(" Ecuacion: {}".format(ecuacion))
9
10    print()
11    print(" Paso 1: Calcular discriminante")
12    print(" D = b^2 - 4ac")
13    valor_b2 = coef_b ** 2
14    valor_4ac = 4 * coef_a * coef_c
15    print(" D = ({} )^2 - 4({})({})".format(coef_b, coef_a, coef_c))
16    print(" D = {} - {} = {}".format(valor_b2, valor_4ac, discriminante))
17
18    if discriminante > 0:
19        print(" D > 0 => Dos raices reales distintas")
20    elif discriminante == 0:
21        print(" D = 0 => Dos raices reales iguales")
22    else:
23        print(" D < 0 => Dos raices complejas conjugadas")
24
25    print()
26    print(" Paso 2: Aplicar formula general")
27    print(" x = (-b +/- sqrt(D)) / (2a)")
28    denominador = 2 * coef_a
29    print(" x = (-({}) +/- sqrt({})) / (2*{})".format(coef_b, discriminante, coef_a))
30    print(" x = ({} +/- {}) / {}".format(-coef_b, formatear_raiz(raiz_discriminante),
31        denominador))
32
33    print()
34    print(" Paso 3: Obtener raices")
35    print(" x1 = {}".format(formatear_raiz(raiz_1)))
36    print(" x2 = {}".format(formatear_raiz(raiz_2)))
```

Donde para esto, al momento de entrar en la funcion, se contruye la ecuacion cuadratica en formato de texto utilizando la funcion `construir_ecuacion_cuadratica`,

donde en los primeros print, se imprime la ecuacion con el uso de la funcion `.format`, la cual nos permite insertar el texto de la ecuacion, despues se imprime el paso 1, el cual tiene como objetivo imprimir el discriminante, para esto se calcula el valor de b^2 y $4ac$ para mostrarlo en la terminal, y despues se muestra el resultado del discriminante, y se hace una pequeña interpretacion de lo que significa el valor del discriminante, si es mayor a cero, igual a cero



o menor a cero. Después de esto se muestra el paso 2, el cual es aplicar la fórmula general, donde esta misma hace uso de los valores pasados en la cabecera de la función, y también se vuelve a usar `.format` el cual en este caso nos permite mostrar la fórmula con los valores de los coeficientes, y el resultado del discriminante, y por último se muestra el paso 3, donde se muestran las raíces obtenidas, utilizando la función `formatear_raiz` para mostrar las raíces de una forma más legible.

La función `formatear_raiz` recibe como parámetro `raiz_compleja`, que es el valor de una raíz que puede ser real o compleja. En la primera línea dentro de la función se evalúa la condición `if abs(raiz_compleja.imag) < 1e-9:`, donde se toma el valor absoluto de la parte imaginaria para verificar si es prácticamente cero. Si esta condición se cumple, entonces la siguiente línea `return "{:.4f}".format(raiz_compleja.real)` devuelve únicamente la parte real con formato de 4 decimales, lo cual permite que una raíz que no tiene imaginario este en el formato correspondiente para imprimirla en la terminal correctamente.

Después, si la condición no se cumple, entonces se pasa al `else`, lo que significa que la raíz sí tiene parte imaginaria significativa. En la línea `signo = "+" if raiz_compleja.imag >= 0 else "-"` se determina el signo que se va a mostrar entre la parte real y la parte imaginaria, usando un operador ternario para elegir `+` cuando la parte imaginaria es positiva o cero, y `-` cuando es negativa. Finalmente, en la última línea del `return`, se construye el formato de número complejo, mostrando la parte real con 4 decimales, después el signo calculado, y al final el resultado se ponga en formato $a + bi$ o $a - bi$ a la salida.

Listing 3: Funciones para formatear matrices y raíces

```
1 def formatear_raiz(raiz_compleja):
2     if abs(raiz_compleja.imag) < 1e-9:
3         return "{:.4f}".format(raiz_compleja.real)
4     else:
5         signo = "+" if raiz_compleja.imag >= 0 else "-"
6         return "{:.4f} {} {:.4f}i".format(raiz_compleja.real, signo, abs(raiz_compleja.
            imag))
```

2.2. Explicación ecuación 3er orden:

Ahora bien, para resolver una ecuación cúbica de tercer orden, tenemos la siguiente fórmula general:



$$ax^3 + bx^2 + cx + d = 0 \quad (3)$$

donde a , b , c y d son los coeficientes de la ecuación, y para este caso se utiliza el método de Cardano, el cual transforma la ecuación original a una forma más fácil de ver la ecuación, tanto para el programa como para nosotros.

Antes de pasar al código, es importante explicar cómo funciona el método de Cardano, con un ejemplo:

Supongamos que tenemos lo siguiente:

$$2x^3 - 4x^2 + 3x - 6 = 0 \quad (4)$$

Se hacen uso de las siguientes ecuaciones auxiliares:

$$t^3 + pt + q = 0 \quad (5)$$

Donde p y q son nuevos parámetros que se calculan a partir de los coeficientes originales, y esta forma de la ecuación es mucho más fácil de resolver utilizando el método de Cardano, el cual se basa en encontrar dos valores u y v que satisfacen las siguientes condiciones:

$$u^3 + v^3 = -q \quad (6)$$

$$uv = -\frac{p}{3} \quad (7)$$

Con estos valores, se pueden obtener las raíces de la ecuación original utilizando la siguiente fórmula:

$$t = u + v \quad (8)$$

$$x = t - \frac{b}{3a} \quad (9)$$

Calculamos los parámetros p y q usando los coeficientes originales $a = 2$, $b = -4$, $c = 3$, $d = -6$:

$$p = \frac{3ac - b^2}{3a^2} = \frac{3(2)(3) - (-4)^2}{3(2)^2} = \frac{18 - 16}{12} = \frac{2}{12} = \frac{1}{6} \quad (10)$$

$$q = \frac{2b^3 - 9abc + 27a^2d}{27a^3} = \frac{2(-4)^3 - 9(2)(-4)(3) + 27(4)(-6)}{27(8)} = \quad (11)$$



$$\frac{-128 + 216 - 648}{216} = \frac{-560}{216} = -\frac{70}{27} \quad (12)$$

Con estos valores, la ecuacion simplificada o deprimida queda:

$$t^3 + \frac{1}{6}t - \frac{70}{27} = 0 \quad (13)$$

Ahora calculamos el discriminante de Cardano:

$$\Delta = \left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3 = \left(-\frac{35}{27}\right)^2 + \left(\frac{1}{18}\right)^3 = \frac{1225}{729} + \frac{1}{5832} = \frac{9800 + 1}{5832} = \frac{9801}{5832} = \frac{121}{72} \quad (14)$$

Como $\Delta > 0$, la ecuacion tiene una raiz real y dos raices complejas conjugadas. Calculamos u y v :

En caso de que $\Delta < 0$, se tendria que calcular las raices cubicas de numeros complejos, lo cual es mas complicado, pero en este caso, como $\Delta > 0$, se pueden calcular las raices cubicas de numeros reales, lo cual es mas sencillo.

$$u = \sqrt[3]{-\frac{q}{2} + \sqrt{\Delta}} = \sqrt[3]{\frac{35}{27} + \frac{11\sqrt{2}}{12}} \approx 1.3738 \quad (15)$$

$$v = \sqrt[3]{-\frac{q}{2} - \sqrt{\Delta}} = \sqrt[3]{\frac{35}{27} - \frac{11\sqrt{2}}{12}} \approx -0.0404 \quad (16)$$

La primera raiz se obtiene directamente:

$$t_1 = u + v \approx 1.3738 + (-0.0404) \approx 1.3333 \quad (17)$$

$$x_1 = t_1 - \frac{b}{3a} = 1.3333 - \frac{-4}{6} = 1.3333 + 0.6667 = 2.0000 \quad (18)$$

Para obtener las otras dos raices, se utilizan las raices cubicas de la unidad. Estas son las tres soluciones de $z^3 = 1$, que se obtienen con la formula $\omega_k = e^{2\pi i k/3}$ para $k = 0, 1, 2$. La primera ($k = 0$) es simplemente 1 (que ya usamos para t_1), y las otras dos son:

$$\omega = e^{2\pi i/3} = \cos\left(\frac{2\pi}{3}\right) + i \sin\left(\frac{2\pi}{3}\right) = -\frac{1}{2} + \frac{\sqrt{3}}{2}i \approx -0.5 + 0.8660i \quad (19)$$

$$\omega^2 = e^{4\pi i/3} = \cos\left(\frac{4\pi}{3}\right) + i \sin\left(\frac{4\pi}{3}\right) = -\frac{1}{2} - \frac{\sqrt{3}}{2}i \approx -0.5 - 0.8660i \quad (20)$$



Estas raíces de la unidad permiten generar las tres soluciones de la cubica a partir de u y v . Para la segunda raíz, calculamos cada producto:

$$\omega \cdot u = (-0.5 + 0.8660i)(1.3738) = -0.6869 + 1.1897i \quad (21)$$

$$\omega^2 \cdot v = (-0.5 - 0.8660i)(-0.0404) = 0.0202 + 0.0350i \quad (22)$$

$$t_2 = \omega u + \omega^2 v = (-0.6869 + 1.1897i) + (0.0202 + 0.0350i) = -0.6667 + 1.2247i \quad (23)$$

$$x_2 = t_2 - \frac{b}{3a} = (-0.6667 + 1.2247i) - (-0.6667) = 0 + 1.2247i \approx 1.2247i \quad (24)$$

Ahora para la tercera raíz, se intercambian ω y ω^2 :

$$\omega^2 \cdot u = (-0.5 - 0.8660i)(1.3738) = -0.6869 - 1.1897i \quad (25)$$

$$\omega \cdot v = (-0.5 + 0.8660i)(-0.0404) = 0.0202 - 0.0350i \quad (26)$$

$$t_3 = \omega^2 u + \omega v = (-0.6869 - 1.1897i) + (0.0202 - 0.0350i) = -0.6667 - 1.2247i \quad (27)$$

$$x_3 = t_3 - \frac{b}{3a} = (-0.6667 - 1.2247i) - (-0.6667) = 0 - 1.2247i \approx -1.2247i \quad (28)$$

Resultado final: Las tres raíces de $2x^3 - 4x^2 + 3x - 6 = 0$ son:

$$x_1 = 2, \quad x_2 = i\sqrt{\frac{3}{2}} \approx 1.2247i, \quad x_3 = -i\sqrt{\frac{3}{2}} \approx -1.2247i \quad (29)$$

Caso $\Delta < 0$ (tres raíces reales):

Cuando el discriminante de Cardano es negativo, la ecuación tiene tres raíces reales distintas. En este caso, los valores dentro de las raíces cúbicas de u y v son números complejos (aunque el resultado final sea real), lo cual se conoce como *casus irreducibilis*. Para evitar trabajar con aritmética compleja innecesaria, se utiliza el **metodo trigonométrico**, que calcula las raíces directamente con funciones trigonométricas:

$$t_k = 2\sqrt{-\frac{p}{3}} \cdot \cos\left(\frac{1}{3} \arccos\left(\frac{-q/2}{\sqrt{(-p/3)^3}}\right) - \frac{2\pi k}{3}\right), \quad k = 0, 1, 2 \quad (30)$$

Ejemplo: Consideremos la ecuación $x^3 - 5x^2 + 2x + 8 = 0$ con $a = 1$, $b = -5$, $c = 2$, $d = 8$.

Calculamos p y q :

$$p = \frac{3(1)(2) - (-5)^2}{3(1)^2} = \frac{6 - 25}{3} = -\frac{19}{3} \quad (31)$$



$$q = \frac{2(-5)^3 - 9(1)(-5)(2) + 27(1)^2(8)}{27(1)^3} = \frac{-250 + 90 + 216}{27} = \frac{56}{27} \quad (32)$$

El discriminante de Cardano es:

$$\Delta = \left(\frac{28}{27}\right)^2 + \left(-\frac{19}{9}\right)^3 = \frac{784}{729} - \frac{6859}{729} = -\frac{6075}{729} = -\frac{25}{3} \approx -8.3333 \quad (33)$$

Como $\Delta < 0$, aplicamos el metodo trigonometrico. Primero calculamos los valores necesarios:

$$r = 2\sqrt{-\frac{p}{3}} = 2\sqrt{\frac{19}{9}} \approx 2.9059 \quad (34)$$

$$\theta = \arccos\left(\frac{-q/2}{\sqrt{(-p/3)^3}}\right) = \arccos\left(\frac{-28/27}{\sqrt{6859/729}}\right) = \arccos(-0.3381) \approx 109.76^\circ \quad (35)$$

Ahora obtenemos las tres raices:

$$t_1 = 2.9059 \cdot \cos\left(\frac{109.76^\circ}{3}\right) = 2.9059 \cdot \cos(36.59^\circ) \approx 2.3333 \quad (36)$$

$$x_1 = t_1 - \frac{b}{3a} = 2.3333 - (-1.6667) = 4 \quad (37)$$

$$t_2 = 2.9059 \cdot \cos\left(\frac{109.76^\circ - 360^\circ}{3}\right) = 2.9059 \cdot \cos(-83.41^\circ) \approx 0.3333 \quad (38)$$

$$x_2 = t_2 - \frac{b}{3a} = 0.3333 - (-1.6667) = 2 \quad (39)$$

$$t_3 = 2.9059 \cdot \cos\left(\frac{109.76^\circ - 720^\circ}{3}\right) = 2.9059 \cdot \cos(-203.41^\circ) \approx -2.6667 \quad (40)$$

$$x_3 = t_3 - \frac{b}{3a} = -2.6667 - (-1.6667) = -1 \quad (41)$$

Resultado: Las tres raices reales de $x^3 - 5x^2 + 2x + 8 = 0$ son $x_1 = 4$, $x_2 = 2$ y $x_3 = -1$.

2.2.1. Explicacion del codigo para la resolucion de ecuacion de 3 orden:

La primera funcion que se define es `raiz_cubica_compleja`, el cual fragmento de codigo es el siguiente:

Listing 4: Función para calcular la raíz cúbica de un número complejo

```
1 def raiz_cubica_compleja(numero_complejo):
2     modulo, angulo = cmath.polar(numero_complejo)
3     return cmath.rect(modulo ** (1 / 3), angulo / 3)
```



Esta funcion recibe un numero complejo como argumento. En la primera linea `modulo`, `angulo = cmath.polar(numero_complejo)` se convierte el numero complejo a su representacion en coordenadas polares, obteniendo su modulo y su angulo de fase. Despues, en la segunda linea, se utiliza `cmath.rect(modulo ** (1 / 3), angulo / 3)` para reconstruir la raiz cubica en forma rectangular, donde el modulo se eleva a la potencia $1/3$ y el angulo se divide entre 3, lo cual corresponde matematicamente a la operacion de raiz cubica en el plano complejo y permite obtener el resultado correcto incluso cuando el argumento es un numero negativo o complejo.



La funcion principal del archivo es `resolver_cubica`, cuyo codigo es el siguiente:

Listing 5: Función principal para resolver la ecuación cúbica

```
1 def resolver_cubica(coef_a, coef_b, coef_c, coef_d, mostrar_pasos=False):
2     if coef_a == 0:
3         return resolver_cuadratica(coef_b, coef_c, coef_d, mostrar_pasos)
4
5     parametro_p = (3 * coef_a * coef_c - coef_b ** 2) / (3 * coef_a ** 2)
6     parametro_q = (2 * coef_b ** 3 - 9 * coef_a * coef_b * coef_c + 27 * coef_a ** 2 *
7         coef_d) / (27 * coef_a ** 3)
8
9     discriminante_cardano = (parametro_q / 2) ** 2 + (parametro_p / 3) ** 3
10
11     valor_u = raiz_cubica_compleja(-parametro_q / 2 + cmath.sqrt(discriminante_cardano))
12     valor_v = raiz_cubica_compleja(-parametro_q / 2 - cmath.sqrt(discriminante_cardano))
13
14     raiz_unidad_1 = complex(-0.5, math.sqrt(3) / 2)
15     raiz_unidad_2 = complex(-0.5, -math.sqrt(3) / 2)
16
17     solucion_t1 = valor_u + valor_v
18     solucion_t2 = raiz_unidad_1 * valor_u + raiz_unidad_2 * valor_v
19     solucion_t3 = raiz_unidad_2 * valor_u + raiz_unidad_1 * valor_v
20
21     offset_cardano = coef_b / (3 * coef_a)
22     raiz_1 = solucion_t1 - offset_cardano
23     raiz_2 = solucion_t2 - offset_cardano
24     raiz_3 = solucion_t3 - offset_cardano
25
26     raices = (raiz_1, raiz_2, raiz_3)
27
28     if mostrar_pasos:
29         mostrar_proceso_cubica(coef_a, coef_b, coef_c, coef_d,
30             parametro_p, parametro_q, discriminante_cardano,
31             valor_u, valor_v, raices)
32
33     return raices, None
```

La funcion recibe los cuatro coeficientes de la ecuacion cubica y el parametro opcional `mostrar_pasos`. En la primera linea se verifica si `coef_a == 0`, lo cual indicaria que la ecuacion en realidad no es cubica sino cuadratica, y en ese caso se delega directamente a `resolver_cuadratica(coef_b, coef_c, coef_d, mostrar_pasos)` devolviendo su resultado sin continuar.

Si la ecuacion si es cubica, entonces se calculan los parametros `parametro_p` y `parametro_q` usando las formulas del metodo de Cardano para la forma deprimida. Despues se calcula `discriminante_cardano = (parametro_q / 2) ** 2 + (parametro_p / 3) ** 3`, y con



ese valor se obtienen `valor_u` y `valor_v` usando `cmath.sqrt` para la raíz cuadrada compleja y la función `raiz_cubica_compleja` que se definió antes.

A continuación se definen `raiz_unidad_1 = complex(-0.5, math.sqrt(3) / 2)` y `raiz_unidad_2 = complex(-0.5, -math.sqrt(3) / 2)`, que son las dos raíces cúbicas complejas de la unidad y permiten construir las tres soluciones de la forma deprimida: `solucion_t1`, `solucion_t2` y `solucion_t3`. Después se calcula `offset_cardano = coef_b / (3 * coef_a)` y se resta a cada solución para regresar de la variable auxiliar t a la variable original x , obteniendo `raiz_1`, `raiz_2` y `raiz_3`. Estas tres raíces se agrupan en la tupla `raices`, y si `mostrar_pasos` es verdadero, se llama a `mostrar_proceso_cubica` con todos los valores calculados para imprimir el procedimiento completo; finalmente la función retorna `raices`, `None`.

La función `mostrar_proceso_cubica` se encarga de imprimir en la terminal el desarrollo paso a paso del método de Cardano:

Listing 6: Función para mostrar el proceso de resolución de la ecuación cúbica

```
1 def mostrar_proceso_cubica(coef_a, coef_b, coef_c, coef_d,
2                             parametro_p, parametro_q, discriminante_cardano,
3                             valor_u, valor_v, raices):
4     ecuacion = construir_ecuacion_cubica(coef_a, coef_b, coef_c, coef_d)
5     print()
6     imprimir_separador()
7     print(" PROCESO DE CALCULO (Metodo de Cardano)")
8     imprimir_separador()
9     print(" Ecuacion: {}".format(ecuacion))
10
11     print()
12     print(" Paso 1: Reducir a forma deprimida t^3 + pt + q = 0")
13     print(" p = (3ac - b^2) / (3a^2)")
14     print(" p = (3*{}*{} - ({}^2) / (3*({}^2)".format(coef_a, coef_c, coef_b, coef_a))
15     print(" p = {}".format(parametro_p))
16     print()
17     print(" q = (2b^3 - 9abc + 27a^2*d) / (27a^3)")
18     print(" q = {}".format(parametro_q))
19
20     print()
21     print(" Paso 2: Calcular discriminante de Cardano")
22     print(" D = (q/2)^2 + (p/3)^3")
23     print(" D = ({}^2 + ({}^3".format(parametro_q / 2, parametro_p / 3))
24     print(" D = {}".format(discriminante_cardano))
25
26     print()
27     print(" Paso 3: Calcular u y v")
28     print(" u = cbrt(-q/2 + sqrt(D))")
29     print(" u = {}".format(formatear_raiz(valor_u)))
```



```
30     print("      v = cbrt(-q/2 - sqrt(D))")
31     print("      v = {}".format(formatear_raiz(valor_v)))
32
33     print()
34     print(" Paso 4: Revertir sustitucion (x = t - b/(3a))")
35     offset = coef_b / (3 * coef_a)
36     print("      Offset = b/(3a) = {}/(3*{}) = {:.4f}".format(coef_b, coef_a, offset))
37
38     print()
39     print(" Paso 5: Raices obtenidas")
40     for indice, raiz in enumerate(raices):
41         print("      x{} = {}".format(indice + 1, formatear_raiz(raiz)))
```

La funcion recibe como parametros todos los coeficientes originales, los valores intermedios del metodo de Cardano y las raices ya calculadas. En la primera linea se construye la ecuacion en formato de texto usando `construir_ecuacion_cubica`, y despues se imprime el encabezado del procedimiento utilizando `imprimir_separador()` para darle formato visual a la salida en terminal.

En el paso 1 se muestran las formulas y los valores numericos de los parametros p y q , que son los que permiten transformar la ecuacion cubica original a la forma deprimida $t^3 + pt + q = 0$, lo cual es una simplificacion necesaria para poder aplicar el metodo de Cardano. Las lineas de `print` usan `.format` para sustituir los coeficientes en la formula y mostrar tanto la expresion simbolica como el valor calculado. En el paso 2 se imprime el discriminante de Cardano $D = (q/2)^2 + (p/3)^3$, mostrando tambien la sustitucion numerica con los valores de `parametro_q` y `parametro_p` antes de presentar el resultado. En el paso 3 se imprime el valor de u y v usando `formatear_raiz` para presentarlos correctamente. En el paso 4 se calcula localmente el `offset = coef_b / (3 * coef_a)`, que es la correccion que se aplica para revertir la sustitucion y regresar de la variable auxiliar t a la variable original x . Finalmente, en el paso 5 se recorren las raices con `enumerate` para imprimirlas numeradas, tambien usando `formatear_raiz`.

2.3. Explicacion de las llamadas a funciones main.py:

La primera funcion que se define en el archivo principal es `preguntar_mostrar_proceso`, cuyo codigo es el siguiente:

Listing 7: Función para preguntar si se muestra el proceso

```
1 def preguntar_mostrar_proceso():
2     respuesta = input("  Mostrar proceso? (s/n): ")
```



```
3 return respuesta.lower() == 's'
```

Esta función se encarga de preguntarle al usuario si quiere ver el proceso de como se resolvió la ecuación. En la primera línea se usa `input` para mostrar el mensaje y guardar lo que el usuario escriba en la variable `respuesta`. Después, en el `return`, se convierte la respuesta a minúsculas con `.lower()` y se compara con la letra `'s'`, de tal forma que si el usuario escribe `'s'` o `'S'`, la función regresa `True`, y en cualquier otro caso regresa `False`.

La siguiente función es `leer_coeficientes`, que se encarga de pedirle al usuario los coeficientes de la ecuación:

Listing 8: Función para leer los coeficientes

```
1 def leer_coeficientes(nombres):
2     print()
3     imprimir_separador()
4     print("  Ingresar los coeficientes:")
5     imprimir_separador()
6     coeficientes = []
7     for nombre in nombres:
8         valor = float(input(" {} = ".format(nombre)))
9         coeficientes.append(valor)
10    return coeficientes
```

Esta función recibe como parámetro `nombres`, que es una lista con los nombres de los coeficientes que se van a pedir, por ejemplo `[.a", "b", "c"]` para una ecuación cuadrática o `[.a", "b", "c", "d"]` para una cúbica. Primero se imprime un separador y un mensaje indicando que se van a pedir los coeficientes. Después se crea una lista vacía llamada `coeficientes`, y con un ciclo `for` se recorre cada nombre de la lista, donde en cada vuelta se le pide al usuario que ingrese el valor de ese coeficiente usando `input` con `.format` para mostrar el nombre correspondiente, se convierte a número decimal con `float` y se agrega a la lista con `.append`. Al final se regresa la lista con todos los coeficientes ingresados.

Después se define la función `manejar_ecuacion_cuadratica`, que es la encargada de juntar todo lo necesario para resolver una ecuación de segundo grado:

Listing 9: Función para manejar la ecuación cuadrática

```
1 def manejar_ecuacion_cuadratica():
2     try:
3         coeficientes = leer_coeficientes(["a", "b", "c"])
4         coef_a, coef_b, coef_c = coeficientes
5
```



```
6     mostrar_pasos = preguntar_mostrar_proceso()
7
8     raices, error = resolver_cuadratica(coef_a, coef_b, coef_c, mostrar_pasos)
9
10    if error:
11        print(error)
12    else:
13        imprimir_resultados(raices)
14 except ValueError:
15    print(" Error: Ingresa unicamente valores numericos.")
```

En esta funcion, lo primero que se hace es llamar a `leer_coeficientes` pasandole la lista `["a", "b", "c"]`, para que le pida al usuario los tres coeficientes de la ecuacion cuadratica. Estos valores se desempaquetan en las variables `coef_a`, `coef_b` y `coef_c`. Despues se llama a `preguntar_mostrar_proceso` para saber si el usuario quiere ver el procedimiento paso a paso, y el resultado se guarda en `mostrar_pasos`. Con estos datos se llama a `resolver_cuadratica`, que regresa dos cosas: las raices y un posible error. Si hubo error (por ejemplo que $a = 0$), se imprime el mensaje de error, y si no, se imprimen las raices con `imprimir_resultados`. Todo esto esta dentro de un bloque `try-except` para que si el usuario ingresa algo que no sea un numero, se atrape el error de tipo `ValueError` y se muestre un mensaje indicando que solo se aceptan valores numericos.

De forma muy similar, se tiene la funcion `manejar_ecuacion_cubica`:

Listing 10: Función para manejar la ecuación cúbica

```
1 def manejar_ecuacion_cubica():
2     try:
3         coeficientes = leer_coeficientes(["a", "b", "c", "d"])
4         coef_a, coef_b, coef_c, coef_d = coeficientes
5
6         mostrar_pasos = preguntar_mostrar_proceso()
7
8         raices, error = resolver_cubica(coef_a, coef_b, coef_c, coef_d, mostrar_pasos)
9
10    if error:
11        print(error)
12    else:
13        imprimir_resultados(raices)
14 except ValueError:
15    print(" Error: Ingresa unicamente valores numericos.")
```

Esta funcion sigue la misma logica que la anterior, con la diferencia de que ahora se piden cuatro coeficientes `["a", "b", "c", "d"]` en lugar de tres, ya que una ecuacion cubica tiene



un termino mas. Los coeficientes se desempaquetan en `coef_a`, `coef_b`, `coef_c` y `coef_d`, y despues se llama a `resolver_cubica` en lugar de `resolver_cuadratica`. El manejo de errores y la impresion de resultados funcionan exactamente igual que en el caso cuadratico.

Por ultimo, se tiene la funcion `main`, que es el punto de entrada del programa y se encarga de mostrar el menu y dirigir al usuario a la opcion que elija:

Listing 11: Función principal main

```
1 def main():
2     while True:
3         imprimir_menu()
4
5         opcion = input("\n Opcion: ")
6
7         if opcion == '0':
8             print()
9             imprimir_separador()
10            print(" Saliendo del programa...")
11            imprimir_separador()
12            break
13        elif opcion == '2':
14            manejar_ecuacion_cuadratica()
15        elif opcion == '3':
16            manejar_ecuacion_cubica()
17        else:
18            print(" Opcion no valida. Intenta de nuevo.")
19
20
21 main()
```

La funcion `main` utiliza un ciclo `while True` para que el programa se mantenga corriendo hasta que el usuario decida salir. En cada vuelta del ciclo, primero se imprime el menu con `imprimir_menu()` y despues se le pide al usuario que ingrese una opcion con `input`. Si el usuario escribe '0', se imprime un mensaje de despedida y se usa `break` para salir del ciclo y terminar el programa. Si escribe '2', se llama a `manejar_ecuacion_cuadratica`, y si escribe '3', se llama a `manejar_ecuacion_cubica`. En caso de que el usuario escriba cualquier otra cosa, se le muestra un mensaje indicando que la opcion no es valida y el ciclo vuelve a empezar mostrando el menu de nuevo. Al final del archivo, la linea `main()` es la que ejecuta todo el programa al momento de correr el archivo.



3. Resultados:



4. Conclusiones:

- Espinoza Matamoros Percival Ulises:
- Flores Colin Victor Jaziel:
- Garcia Cortes Adolfo de Jesus:
- Lara Hernandez Angel Husiel:
- Lugo Manzano Rodrigo:



Referencias

- Arm Ltd. (2020). *Arm Cortex-M0+ Processor Datasheet* (Datasheet) (Documento PDF. Año aproximado). Arm Limited.
- Raspberry Pi Ltd. (2025, 20 de febrero). *RP2040 Datasheet: A microcontroller by Raspberry Pi*. Ver. build-version: 3184e62-clean. Consultado el 27 de febrero de 2026, desde <https://pip-assets.raspberrypi.com/categories/814-rp2040/documents/RP-008371-DS-1-rp2040-datasheet.pdf>
- Raspberry Pi Ltd. (2026). *Microcontroller chips - Raspberry Pi Documentation*. Consultado el 26 de febrero de 2026, desde <https://www.raspberrypi.com/documentation/microcontrollers/microcontroller-chips.html>
- Transfer Multisort Elektronik (TME). (2024). *GPIO - definición — Glosario TME*. Consultado el 27 de febrero de 2026, desde <https://www.tme.com/mx/es/news/library-articles/glossary/page/61888/gpio-general-purpose-inputoutput-definicion/> Artículo de glosario técnico.